



Artificial Intelligence Software, Lecture 1

Introduction

Intro to MLOps



Center for
Cognitive
Modeling

About Me (1)



PhD Oleg Bulichev

Mail: bulichev.ov@mipt.ru

TG: [@Lupasic](#)

Office Hours: By request

About Me (2)

Education

- *Bachelor* — BMSTU (МГТУ им Н.Э. Баумана) «CAD developer»
- *Master & PHD* — Innopolis University (IU) «Robotics»

Job

- *Lead Engineer, Senior Research Fellow* — MIPT, Center for Cognitive Modeling: SLAM and robotics hardware development
- *Associate Professor* — IU: Supervisor



Link to my channel

Course Goal



To obtain an **instrumental foundation** (the **tools**)
so that in subsequent courses
(Intro to ML, DL, RL, etc.)
*you can learn to **build a house***



Course purpose and objectives

Our laboratory works with **AI in its full scope**, with special attention to **robotics**.

Therefore, it's important for us to:

- Not only find solutions as **researchers**
- But also **maintain** and **deploy** them to hardware competently
- Create **demos and web interfaces** with running models for presentations

We will go through the **complete pipeline** and learn its various **tools**:

- Data Management
- Model Selection & Configuration
- Training & Monitoring
- Demo Development
- Optimization & Deployment
- Robotics tools



Course challenges

- Inverse education (first tools, then theory)
- A lot of coding and debugging
- Dive into the "jungle" of tools and frameworks
- Completely new course



Course outline and organization

Three big blocks:

- Tools: Conda, Docker, VibeCoding, Literature Review using AI, ROS (Robotics Operating System), Simulators
- Training pipeline: Data management, Model selection, Training and monitoring
- Deployment pipeline: Demo development, Model optimization, Deployment to hardware



Grading criteria

What will be evaluated on the course

Qz: Quizzes: $4 \times 5 = 20\%$

HWs: Homework assignments: 40%

CP: Course project: $2 \times 15 = 30\%$

FE: Final Exam: 10%

Extra: Slide fixes in Github: 5%

Late policy: -50% of max grade for a task

Scale:

5: 85 — 100%

4: 70 — 84.99%

3: 50 — 69.99%

2: 0 — 49.99% or less than 50% by any criterion.



Quizzes

Purpose: Quizzes help us to understand what you didn't get during the previous lecture.

Format: 5 — 15 *min* closed book on a paper sheet, in the beginning of a lecture.



Course Project

Projects are extended version of homeworks.

Idea for project 1: Build a custom image classifier for 5-10 classes, deploy it on a web server with a simple web interface.

Idea for project 2: Take pre-trained YOLO, label your data, fine-tune, deploy on PC, integrate your camera with robotics simulator.

Lab Goals

To obtain the needed tools for solving the design part of the competition



Agreements among the group (for now)



Pretest

- Dataset
- Model
- Training
- Inference
- Deployment
- Pipeline



Key Terms (1/2)

- **Dataset:** Collection of data used to train or test models
Example: ImageNet, KITTI
- **Model:** Mathematical representation that learns patterns from data
Example: YOLO, ResNet
- **Training:** Process of teaching a model by feeding it data
Example: Using PyTorch to adjust model weights



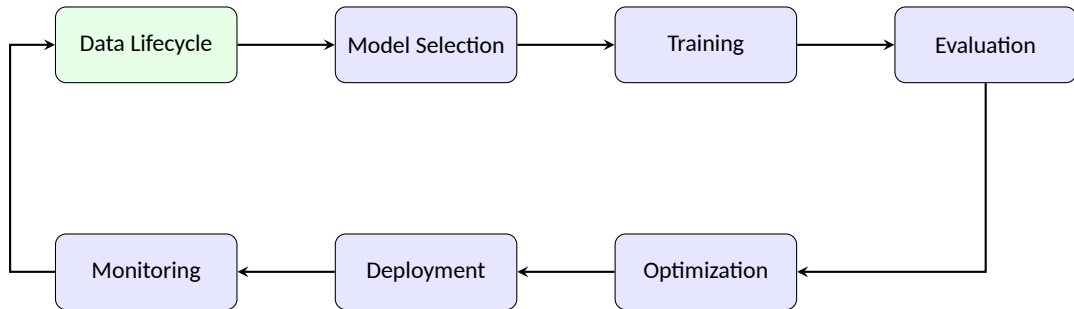
Key Terms (2/2)

- **Inference:** Using a trained model to make predictions on new data
Example: Running YOLO to detect objects in a video stream
- **Deployment:** Making a model available for use in production
Example: Hosting model on a server, deploying to Jetson
- **Pipeline:** Automated sequence of steps from data to deployment
Example: Data → Training → Testing → Deployment



Let's discuss ML pipelines you used/heard before!

Pipeline Overview





1. Data Lifecycle

Garbage in, garbage out

- **Key Steps:** Collection, Annotation (Labeling), Versioning
- **Why it's needed:** A model cannot learn what isn't in the data. Reproducible data ensures reproducible results.
- **Example Task:** Collecting video from a robot, drawing bounding boxes around obstacles, creating a Dataset v1.0 snapshot.



2. Model Selection

Don't reinvent the wheel

- **Key Steps:** Literature review, Selecting pre-trained architectures, Configuring hyperparameters.
- **Why it's needed:** Starting from a pre-trained model (SOTA) saves weeks of compute time and resources.
- **Example Task:** Choosing a lightweight object detector that works well on a small dataset of traffic signs.



3. Training

Scientific approach to experiments

- **Key Steps:** Fine-tuning, Logging metrics/weights, Hyperparameter optimization.
- **Why it's needed:** To understand *which* change improved the model and to be able to reproduce the best result.
- **Example Task:** Running 10 experiments with different learning rates and selecting the one with the lowest loss curve.



4. Evaluation

Proof of Concept (PoC)

- **Key Steps:** Calculating metrics (IoU, Accuracy on test set), Visual testing (Demo UI).
- **Why it's needed:** Dry metrics don't always reflect reality. Stakeholders need to "see" how it works on real examples.
- **Example Task:** Creating a simple web interface where you upload a photo and see the robot's planned path.



5. Optimization

Making it fit the hardware

- **Key Steps:** Format conversion, Quantization (e.g. FP32 $\rightarrow$ INT8), Hardware compilation.
- **Why it's needed:** Python is often too slow for real-time robotics. Robots have limited battery and compute power.
- **Example Task:** Reducing model size by 4x to run at 30 FPS on a robot's onboard computer.



6. Deployment

Integrating the brain into the body

- **Key Steps:** Containerization, Inference Server setup, Hardware integration.
- **Why it's needed:** To run the model reliably in a production environment (outside the developer's laptop), handling errors and restarts.
- **Example Task:** Packaging the model into a container that automatically starts when the robot turns on.



7. Monitoring

The world changes, and so should the model

- **Key Steps:** Metric collection, Alerting, Data Drift detection.
- **Why it's needed:** To know when the model starts failing (e.g., at night, in snow, or due to sensor degradation) *before* users complain.
- **Example Task:** A dashboard showing that the confidence of obstacle detection drops significantly during rain.

Is this pipeline universal?

ML, DL, NLP, CV, RL ... ?

- Does the **Data** → **Model** → **Deploy** cycle apply to all AI fields?
- **Yes!** The high-level stages remain the same.
- **However**, the tools, data formats, and specific challenges ("bottlenecks") differ significantly.
- Key domains we will touch: **Classical ML, Computer Vision (CV), NLP, Reinforcement Learning (RL)**.



Classical ML

Structured Data, Spreadsheets, SQL

- **Data:** Structured (rows/columns). Feature Engineering is crucial (handling missing values, normalization).
- **Model:** Light architectures (Random Forest, XGBoost, Linear Regression).
- **Pipeline Specifics:**
 - Training is fast (minutes/seconds).
 - Explainability is easier (Feature Importance).
 - Deployment is lightweight (often just a serialized pickle file).
- **Example:** Bank credit scoring, Customer churn prediction.



Computer Vision (CV) & NLP

Unstructured Data (Deep Learning)

- **Data:** Images, Video, Text, Audio. Requires huge storage and specialized labeling (bounding boxes, tokens).
- **Model:** Deep Neural Networks (CNNs, Transformers).
- **Pipeline Specifics:**
 - Needs GPUs for training (hours/days).
 - Data Augmentation is key.
 - Transfer Learning (Pre-trained models) is standard.
- **Example:** Face recognition, ChatGPT, Object Detection.



Reinforcement Learning (RL)

The Outlier: Learning by Interaction

- **Data:** Not a static dataset! Data is generated by the Agent interacting with an Environment (Simulation).
- **Model:** Policy Networks (Agents).
- **Pipeline Specifics:**
 - "Training" loop involves simulation steps (millions of frames).
 - **Sim2Real gap:** Hard to transfer from simulation to reality.
 - Reward Engineering instead of Labeling.
- **Example:** Robot walking, AlphaGo, Dota 2 bots.



Reference Materials

TODO

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521